

Hackathon IDU — Guide P4

Création du dépôt Git & infrastructure CI/CD

À faire en tout premier · Durée estimée : 45 min

Contexte

Ce document décrit toutes les actions que P4 doit réaliser pour initialiser le dépôt Git partagé. Ces étapes doivent être complétées et vérifiées AVANT que les autres membres de l'équipe ne commencent à coder.

Prérequis :

- Git installé en local (git --version pour vérifier)
- Compte GitHub avec accès aux pseudo des 4 autres membres
- Python 3.11+ installé (python --version)
- pip disponible

1

Créer le repo GitHub

Sur github.com — avant d'écrire la moindre ligne

Aller sur github.com → bouton vert New repository et remplir :

Nom du repo	audit-idu-qualite
Visibilité	Private
Initialisation	Cocher "Add a README file"
.gitignore	Cocher → choisir Python
Licence	Ne pas cocher (inutile)

Ensuite : Settings → Collaborators → Add people → ajouter les 4 autres membres par leur pseudo GitHub. Ils recevront un email d'invitation.

2

Cloner le repo en local

Sur ta machine

```
git clone https://github.com/TON-PSEUDO/audit-idu-qualite.git
cd audit-idu-qualite
```

3

Créer la structure de dossiers

Une seule commande

```
mkdir -p src/parsers src/rules app tests data output docs
```

Créer les fichiers __init__.py pour que Python reconnaisse les modules :

```
touch src/__init__.py src/parsers/__init__.py src/rules/__init__.py
```

```
touch tests/__init__.py
```

Structure résultante :

```
audit-idu-qualite/
├─ src/
│   ├── parsers/      ← P1 travaille ici
│   └─ rules/         ← P2 travaille ici
├─ app/               ← P3 travaille ici
├─ tests/             ← P4 travaille ici
├─ data/              ← fichiers JSON (lecture seule, non versionnés)
├─ output/            ← résultats générés (non versionnés)
├─ docs/              ← P5 travaille ici
├─ requirements.txt
├─ Makefile
└─ .github/workflows/ci.yml
```

4

Compléter le .gitignore

GitHub en a créé un, il faut l'enrichir

Ouvrir .gitignore et ajouter à la fin :

```
# Outputs générés
output/
htmlcov/
.coverage

# Données sources (partagées autrement, pas sur Git)
data/

# Environnements virtuels
.venv/
venv/

# MacOS
.DS_Store

# IDE
.vscode/
.idea/
```

ATTENTION : le dossier data/ est ignoré. Les fichiers JSON se partagent via Drive ou Discord, pas via Git.

5

Créer requirements.txt

Toutes les dépendances du projet

```
pandas>=2.0
networkx>=3.0
plotly>=5.0
streamlit>=1.35
beautifulsoup4>=4.12
python-dateutil>=2.9
```

```
pytest>=8.0
pytest-cov>=5.0
```

Vérifier que tout s'installe sans erreur :

```
pip install -r requirements.txt
```

6

Créer le Makefile

Raccourcis de commandes pour toute l'équipe

IMPORTANT : les indentations dans un Makefile doivent être des tabulations, pas des espaces.

```
.PHONY: install test lint audit run

install:
    pip install -r requirements.txt

test:
    pytest tests/ -v --cov=src --cov-report=html

lint:
    python -m flake8 src/ tests/ --max-line-length=100

audit:
    python -m src.rules.engine

run:
    streamlit run app/dashboard.py
```

7

Créer le pipeline CI GitHub Actions

.github/workflows/ci.yml

```
mkdir -p .github/workflows
```

Créer le fichier .github/workflows/ci.yml avec ce contenu :

```
name: CI Audit IDU

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
```

```

    run: pip install -r requirements.txt

- name: Lint
  run: python -m flake8 src/ tests/ --max-line-length=100

- name: Run tests
  run: pytest tests/ -v --cov=src --cov-report=html

- name: Upload coverage report
  uses: actions/upload-artifact@v4
  with:
    name: coverage-report
    path: htmlcov/

```

Défi 1 accompli : le rapport de couverture est publié automatiquement comme artifact GitHub à chaque push.

8

Créer un test factice pour valider la CI

La CI doit passer au vert au premier push

Créer tests/test_placeholder.py :

```

def test_placeholder():
    """Test temporaire pour valider que la CI tourne."""
    assert 1 + 1 == 2

```

9

Créer les branches de travail

Une branche par personne

```

git checkout -b develop
git push origin develop

git checkout -b feat/ingestion
git push origin feat/ingestion

git checkout -b feat/rules
git push origin feat/rules

git checkout -b feat/dashboard
git push origin feat/dashboard

git checkout develop

```

Chaque personne travaille sur sa branche. On merge sur develop à chaque fin de session via Pull Request.

10

Premier commit et push

Mettre toute la structure en ligne

```

git checkout develop
git add .
git commit -m "chore: init project structure, CI, requirements, Makefile"
git push origin develop

```

Allez sur GitHub → onglet Actions → vérifiez que la CI passe au vert avant de prévenir l'équipe.

11

Partager avec l'équipe

Message à envoyer dans le chat de groupe

```
# Repo GitHub :
https://github.com/TON-PSEUDO/audit-idu-qualite

# Commandes pour démarrer :
git clone https://github.com/TON-PSEUDO/audit-idu-qualite.git
cd audit-idu-qualite
pip install -r requirements.txt

# Données à placer dans data/ (partagées ci-joint) :
data/MAQUETTE_IDU.json
data/ADECal_IDU3.json
data/ADECal_IDU4.json
data/ADECal_IDU5.json
data/Responsables_modules_IDU.json
data/dependance_sequence_IDU.json
data/Résumé Moodle IDU.html
```

Convention de commits (toute l'équipe)

Chaque commit doit commencer par un préfixe indiquant son type :

Préfixe	Usage	Exemple
feat:	Nouvelle fonctionnalité	feat: add ADE parser
test:	Ajout ou modification de tests	test: add parser unit tests
fix:	Correction de bug	fix: handle INFO931 format
chore:	Maintenance / config	chore: update requirements
docs:	Documentation	docs: add methodology section

Checklist finale avant de prévenir l'équipe

[]	Repo GitHub créé et les 4 membres invités
[]	Structure de dossiers créée (src/, tests/, data/, etc.)
[]	.gitignore contient data/ et output/

[]	requirements.txt créé et pip install réussi
[]	Makefile créé avec make test, make run
[]	.github/workflows/ci.yml créé
[]	Branches feat/ingestion, feat/rules, feat/dashboard créées
[]	Premier commit pushé sur develop
[]	CI verte sur GitHub Actions
[]	Lien du repo + fichiers données partagés à l'équipe